

ADVANCED SCIENTIFIC CALCULATOR USING PYTHON TKINTER

Project Documentation

Created By

Sweety

Technology Stack

- Python 3.x
 - Tkinter GUI Framework
 - Python Math Module
-

1. INTRODUCTION

The Advanced Scientific Calculator is a desktop-based graphical application developed using Python and the Tkinter library. The application is designed to perform both basic arithmetic operations and advanced scientific calculations through an interactive graphical user interface (GUI).

The calculator supports operations such as addition, subtraction, multiplication, division, trigonometric functions, logarithmic calculations, square roots, factorials, exponents, and mathematical constants like Pi (π) and Euler's Number (e).

This project demonstrates the practical implementation of Python programming concepts including functions, variables, loops, event handling, GUI development, and exception handling.

2. OBJECTIVES OF THE PROJECT

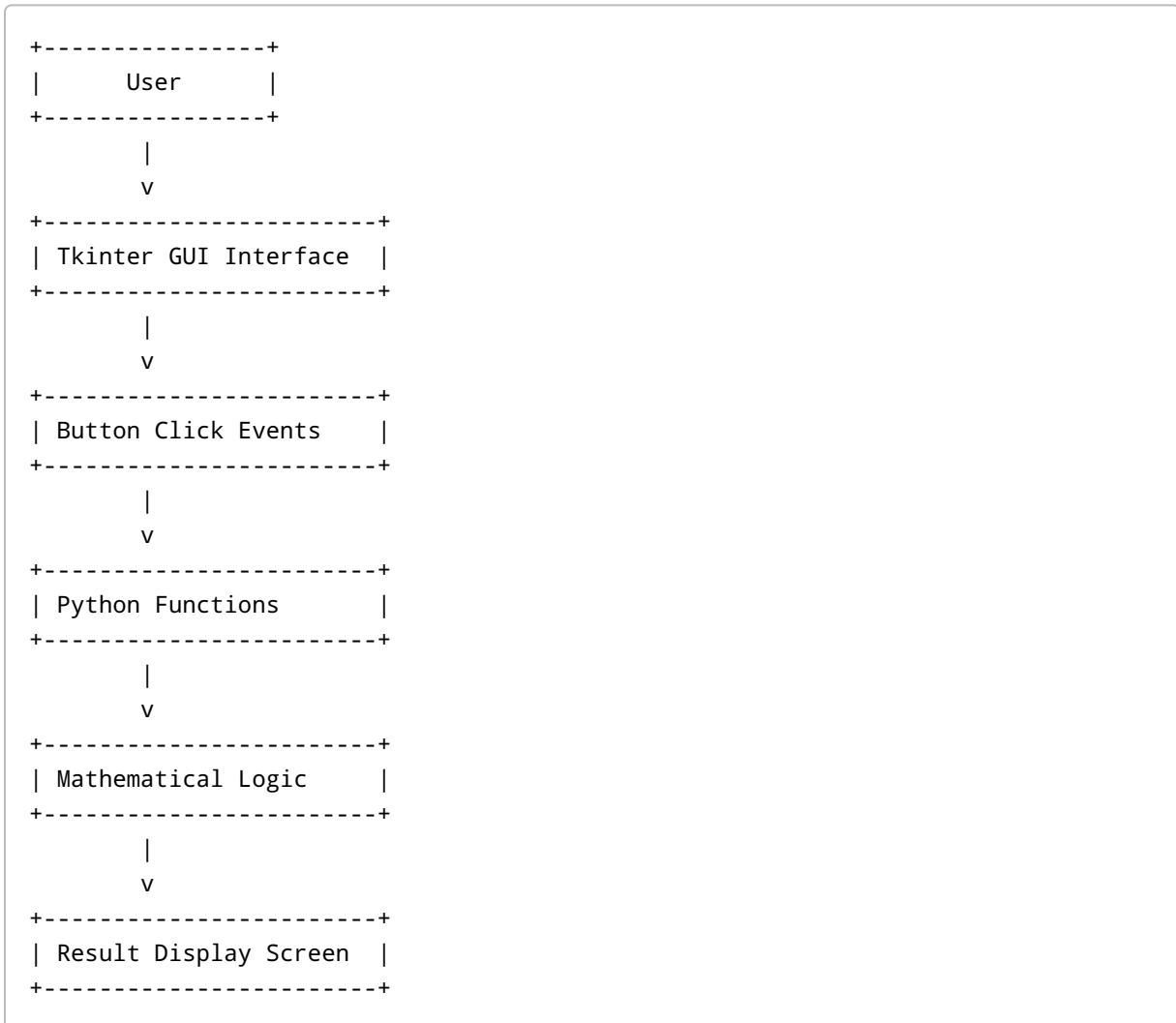
The primary objectives of this project are:

- To develop a user-friendly scientific calculator using Python.
 - To understand GUI development using Tkinter.
 - To implement event-driven programming concepts.
 - To perform scientific calculations using the Math module.
 - To gain practical experience in software design and development.
-

3. PROJECT ARCHITECTURE

The calculator follows an event-driven architecture where user interactions trigger specific functions that process data and update the user interface.

System Architecture Diagram

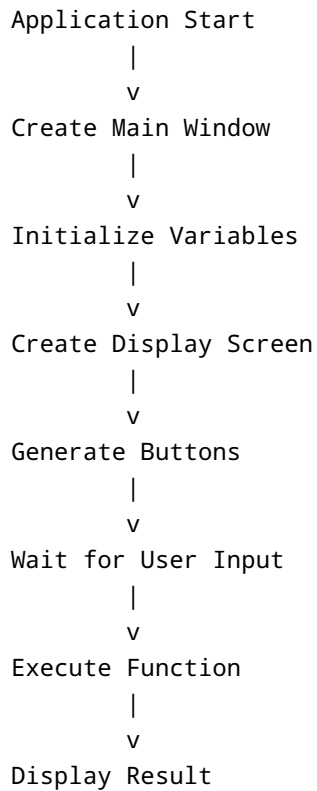


Architecture Explanation

1. The user interacts with the calculator through the GUI.
2. Every button click generates an event.
3. The corresponding Python function is executed.
4. Mathematical calculations are processed.
5. The result is displayed on the calculator screen.

4. APPLICATION WORKFLOW

The execution flow of the calculator is illustrated below:



Workflow Explanation

When the application starts, a calculator window is created. The display screen and buttons are initialized. The application then enters an event loop where it continuously waits for user interaction. Based on the button clicked, the corresponding function executes and updates the display.

5. WINDOW CONFIGURATION

The main application window is created using Tkinter.

Code Snippet

```
root = Tk()

root.title("Advanced Scientific Calculator")
```

```
root.geometry("500x565")

root.resizable(False, False)
```

Explanation

Statement	Description
Tk()	Creates the main application window
title()	Sets the title displayed on the window
geometry()	Defines the width and height of the window
resizable()	Prevents users from resizing the window

6. APPLICATION STATE VARIABLES

The calculator uses two primary variables for managing data.

Code Snippet

```
expression = ""

equation = StringVar()
```

Explanation

Variable	Purpose
expression	Stores the mathematical expression entered by the user
equation	Updates the display screen automatically

Example

If the user enters:

7 + 5

The value stored inside the expression variable becomes:

```
expression = "7+5"
```

The equation variable updates the display accordingly.

7. DISPLAY SCREEN IMPLEMENTATION

The display area is responsible for showing the current expression and calculated results.

Code Snippet

```
entry = Entry(
    root,
    textvariable=equation,
    font=("Arial", 24),
    justify="right"
)
```

Explanation

- Entry widget creates the calculator screen.
 - textvariable links the display to the equation variable.
 - justify="right" aligns numbers like a real calculator.
 - Large font improves readability.
-

8. BUTTON GENERATION MECHANISM

Instead of manually creating every button, the calculator stores all button labels inside a two-dimensional list.

Code Snippet

```
buttons = [
    ['MC', 'MR', 'M+', 'M-', 'C', '∞'],
    ['sin', 'cos', 'tan', 'log', 'ln', '/'],
    ['asin', 'acos', 'atan', 'π', 'e', '*']
]
```

Advantages

- Reduces code complexity.

- Simplifies maintenance.
 - Allows dynamic button generation.
-

9. DYNAMIC BUTTON CREATION USING LOOPS

Buttons are created using nested loops.

Code Snippet

```
for r in range(len(buttons)):
    for c in range(len(buttons[r])):
```

Explanation

The outer loop traverses rows while the inner loop traverses columns. This approach automatically creates all calculator buttons without requiring individual button definitions.

Benefits

- Less repetitive code
 - Better scalability
 - Easier modifications
-

10. BUTTON CLICK PROCESSING

Whenever a user clicks a button, the corresponding value is passed to the `press()` function.

Code Snippet

```
def press(value):
    global expression

    expression += str(value)

    equation.set(expression)
```

Working Example

Before clicking:

```
expression = "7+"
```

User clicks:

```
5
```

After clicking:

```
expression = "7+5"
```

The display updates immediately to show the new expression.